AGILE WAYS OF WORKING

Small increments • Visible progress • Feedback • Adaptation



Deliver useful work in manageable pieces, check progress and respond when needs change.

What Agile means

A practical approach to organising development work

1. Break work down

Large work is divided into smaller pieces that can be understood, implemented and reviewed.

2. Make progress visible

The team can see what is planned, underway and complete.

3. Use feedback

Review results, identify issues and adjust the next piece of work.

4. Communicate

Share progress, next steps and blockers clearly.

5. Adapt

Change priorities or approach when project circumstances change.

Agile and sequential working

The difference is mainly how work is planned and adjusted

REAL PROJECTS
can combine practices

Sequential view

- More work is defined before implementation
- Activities commonly follow a planned sequence
- Changes later in the process may require replanning
- Useful where requirements are stable and predictable

Agile view

- Work is divided into manageable increments
- Progress is reviewed regularly
- Feedback can influence the next increment
- Useful where requirements may change

Why visibility matters

A board is useful when it reflects the real state of work



Scrum in a simple project

A small cycle for planning, coordination and improvement

SPRINT PLANNING

Choose realistic work for the next time-boxed period.

DAILY STAND-UP

Short update: progress • next work • blockers.

SPRINT REVIEW

Look at completed work and collect feedback.

RETROSPECTIVE

Discuss what went well, what was difficult and what to change.

Keep meetings short, useful and focused on coordination — not blame.

Sprint planning

Choose work that can realistically be completed and reviewed

1. **Review priorities**
2. **Select small items**
3. **Check capacity**
4. **Identify dependencies**
5. **Agree the sprint goal**

Example sprint goal: “Complete the task creation feature and verify the main user flows.”

Daily stand-up

A short coordination check — not a long status meeting

DONE

What has been completed?

NEXT

What will be worked on next?

BLOCKED

What is preventing progress?

Keep updates concise. Raise a blocker early instead of waiting.

Example: “Login form is complete. I’m testing error handling next. I’m blocked by the missing API endpoint.”

Retrospective

Improve the way the team works

WHAT WENT WELL

Useful practices to continue

WHAT WAS HARD

Problems, delays or confusion

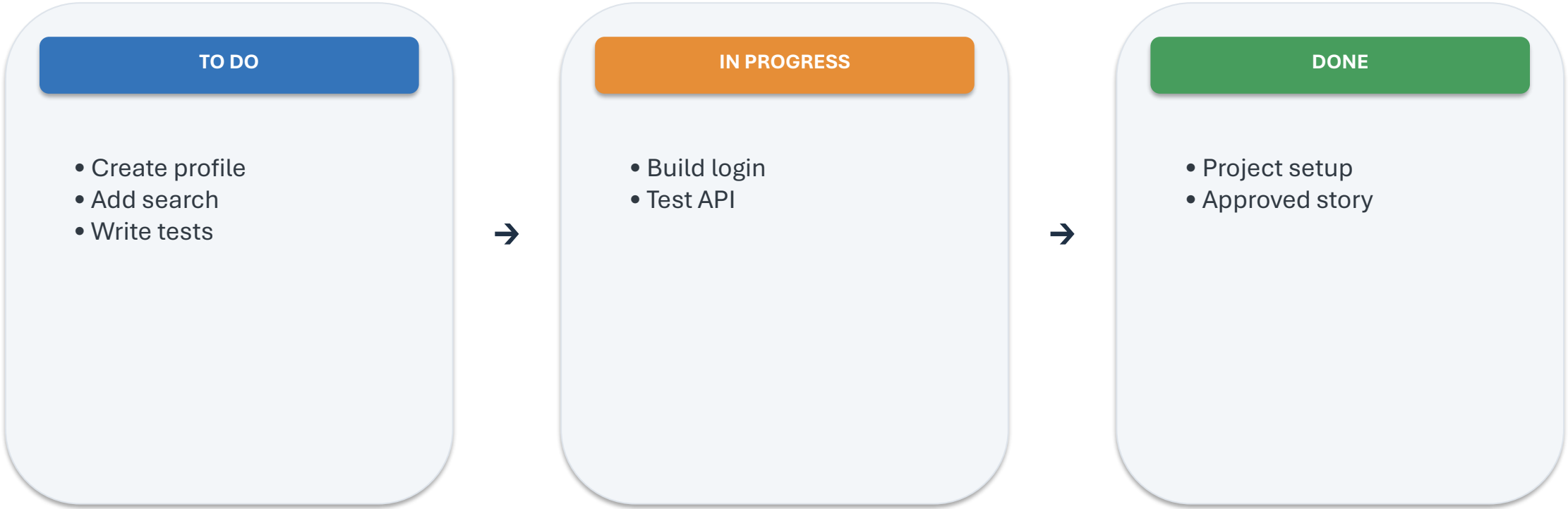
WHAT CHANGES

A practical improvement for the next cycle

Focus on learning and improvement — not assigning blame.

Kanban

Make workflow visible and limit confusion



Avoid starting too many items at once. Too much work in progress hides unfinished work.

User stories

Turn a need into a manageable piece of work

As a [type of user], I want [something], so that [reason/value].

WHO?

The person or role that needs the feature.

WHAT?

The capability or outcome required.

WHY?

The value or reason it matters.

Example: As a learner, I want to mark a task as complete, so that I can track my progress.

Acceptance criteria

Clear conditions that can be checked

Good criteria

✓ Specific
✓ Observable
✓ Testable
✓ Linked to expected behaviour

Example:
• User can select a task by ID.
• Status changes to Completed.
• Updated status appears when tasks are displayed.

Avoid vague wording

✗ “Works properly”
✗ “Easy to use”
✗ “Feature is good”

Instead, define what should happen and how the result can be checked.

Code finished ≠ automatically Done. Evidence should show that the criteria have been met.

Breaking a large story into smaller work

Large story

“Build a complete user account system.”

→

1. Registration
2. Login
3. Password reset
4. Validation
5. Tests
6. Review

Smaller items are easier to implement, test, review and track.

Risks, dependencies and blockers

Three different situations

RISK

A possible future problem.

Example:
“The API may not be ready before the sprint ends.”

DEPENDENCY

Something one item relies on.

Example:
“The dashboard depends on the database table being available.”

BLOCKER

A current obstacle preventing progress.

Example:
“The test environment is unavailable.”

Handling blockers and escalation

Raise issues early and clearly

IDENTIFY What exactly is stopping progress?

CHECK Can it be resolved within your role?

COMMUNICATE Tell the relevant person and explain the impact.

ESCALATE Ask for a decision when it exceeds your authority.

GitHub Projects + Issues

Planning information can connect with repository activity

ISSUE

A clear item of work

Example:
“Add validation to registration form”

Include useful details and acceptance criteria.

PROJECT BOARD

Track status:
To Do → In Progress → Done

Move the item when its real status changes.

EVIDENCE

Connect work to:
- commits
- code changes
- tests
- review comments

Completion should be supported by evidence.

Use the board as a working tool, not just a presentation screen.

AI in Agile work

AI can support planning, but project accuracy stays with the team

AI CAN HELP

- Draft a user story
- Suggest acceptance criteria
- Brainstorm risks
- Organise retrospective notes
- Suggest questions to investigate

CHECK THE OUTPUT

- Does it match the actual project?
- Are assumptions correct?
- Are criteria testable?
- Are risks realistic?
- Has sensitive information been handled appropriately?

FINAL RECORD

Use verified information in the board, Issues, notes and evidence.

AI-generated text should not automatically become the project record.

Agile workflow — one complete cycle

01

NEED

Identify a useful outcome

02

STORY

Write a manageable story

03

CRITERIA

Define observable completion conditions

04

PLAN

Select realistic work

05

BUILD

Implement the solution

06

TEST

Check against criteria

07

REVIEW

Collect feedback

08

ADAPT

Improve the next cycle

The cycle repeats as the project develops.

Quick activity

Plan a small feature using Agile ideas

SCENARIO

A task app needs a feature that allows users to mark a task as completed.

CREATE

1. User story
2. Three acceptance criteria
3. Three Kanban cards
4. One possible risk
5. One possible dependency
6. One possible blocker

DISCUSS

Which item should be done first?

What evidence would show the story is complete?

What should happen if progress is blocked?

Keep the work small, visible, testable and connected to the actual requirement.

Key points

SMALL INCREMENTS

Break large work into manageable pieces.

VISIBILITY

Show what is planned, underway and complete.

FEEDBACK

Review results and use feedback to guide the next step.

COMMUNICATION

Share progress, next work and blockers clearly.

EVIDENCE

Use code and testing to support completion.

IMPROVEMENT

Use retrospectives to make the next cycle better.

Agile is a way of organising and improving work — not just a set of meetings or board columns.

Mini assessment

Check the main concepts

1. What is the purpose of breaking work into small increments?
2. What three updates are normally shared in a stand-up?
3. What is the difference between a risk and a blocker?
4. Why should acceptance criteria be testable?
5. What does a Kanban board make visible?
6. Give one example of a dependency.
7. What is the purpose of a retrospective?
8. How can AI support Agile work without replacing project judgement?

Use examples from the project rather than memorised definitions.